# Advanced Retrieval II: Re-ranking

Modulo 3 · Lezione 5



# Il punto di partenza: quattro righe non prime

| query | chunk giusto | hybrid | chi sta davanti |
|---|---|---|---|
| Cosa copre SRV-02? | listino-catalogo-006 | 2 | la garanzia SRV-03 |
| Quanto tempo ho… contributo… | guida-smart-working-004 | 3 | due chunk vicini della guida |
| …codice sconto: posso restituire? | policy-resi-garanzia-007 | 2 | il chunk degli RMA |
| …si ferma sempre a metà corsa? | ticket-supporto-003 | 2 | un altro ticket del cluster |
| La scrivania non sta ferma… | manuale-lumadesk-013 | — | fuori in tutte e tre |

Le altre otto con un chunk giusto: prime. La tabella è quella che avete compilato la lezione scorsa.

PERCHÉ RRF NON CURA

Su SRV-02 le due ricerche **concordano sul chunk sbagliato**: la fusione premia la concordanza.

Sulle tre concettuali un chunk vicino, in entrambe le liste, **scavalca** il giusto che in BM25 stava in fondo.

La fusione ha solo le posizioni: non guarda i chunk.

> Stasera un **secondo stadio** che guarda i chunk uno per uno, insieme alla domanda.



# Agenda

| | Blocco | Dove | Tempo |
|---|---|---|---|
| | Registrazione a Cohere (facoltativa) | `dashboard.cohere.com` → `.env` | 8′ |
| | Bi-encoder e cross-encoder, i due stadi, Cohere vs LLM | slide | 37′ |
| | Codice spiegato: `rerank_cohere()` (già nel file) | `working/rerank.py` | 12′ |
| | Esercizio: `rerank_llm()` e `cerca_due_stadi()` (TODO 2-3) | `working/rerank.py` | 45′ |
| | Reveal e wrap-up | slide | 18′ |

Un solo file per tutta la serata: `working/rerank.py`, dalla cartella `scripts/` (`python working/rerank.py`). `hybrid.py` è passato in `aikit/` e si importa.



# Registrazione a Cohere (facoltativa)

**FACOLTATIVO — chi vuole provare Cohere fa la chiave adesso**

`dashboard.cohere.com · chiave Trial gratuita · 8 minuti`

si rientra alle slide allo scadere

Cohere ospita il reranker della demo di tra mezz'ora. Chi vuole provarlo sulla propria macchina fa la chiave adesso: è gratuita. Chi preferisce, lavora col secondo backend, la chiave OpenAI che avete già.

1

`dashboard.cohere.com` → account gratuito (email o Google)

2

**API keys** → crea una chiave **Trial** e copiala

3

in `scripts/.env`: `COHERE_API_KEY=…`, accanto a `OPENAI_API_KEY`

LIMITI DEL FREE TIER

**10 chiamate al minuto**, **1.000 al mese**. Il main di stasera aspetta 6 secondi tra una domanda e l'altra.

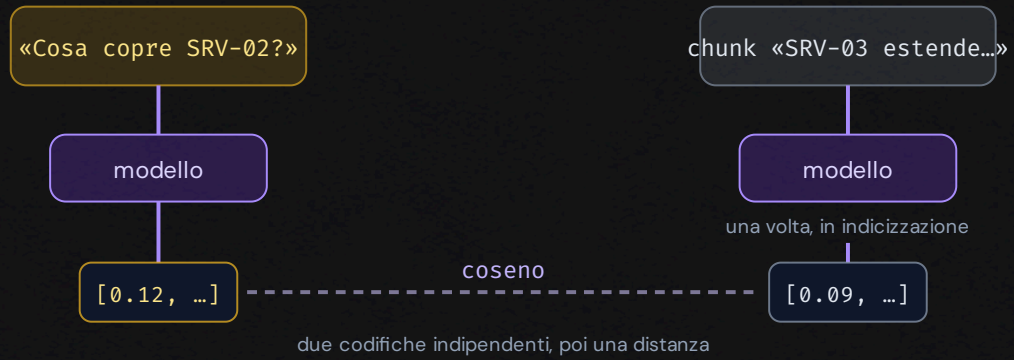
SENZA LA CHIAVE

L'esercizio si fa lo stesso: il main salta la colonna cohere e resta il backend LLM, che è quello che scrivete voi.

La chiave non si incolla nel codice né in chat: solo in `.env`.



# Il bi-encoder: domanda e chunk codificati separatamente



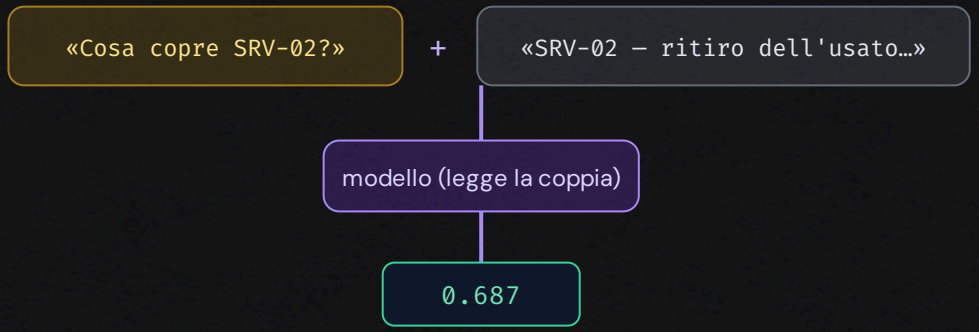

È il retrieval di `rag.recupera()` da Modulo 2 · Lezione 7, col suo nome: **bi-encoder**. BM25 è lo stesso schema in un altro spazio: token contro token, una somma di pesi.

i vettori dei chunk si **pre-calcolano**: regge un milione di chunk

il modello **non vede mai** domanda e chunk insieme

Il vettore di «SRV-03 estende la garanzia» è stato calcolato senza sapere che la domanda sarebbe stata «Cosa copre SRV-02?». Risultato sulla demo: `0.0328` · `0.0320` · `0.0315`, tre score appiccicati.

# Il cross-encoder: domanda e chunk letti insieme




Il **cross-encoder** riceve la coppia in un solo testo e restituisce un numero: quanto *questo* chunk risponde a *questa* domanda.

- mette in relazione SRV-02 della domanda con SRV-02 del chunk, parola contro parola
- distingue cinque ticket che iniziano tutti allo stesso modo
- non produce vettori: **niente da pre-calcolare**

Sulla stessa top-10 della demo: hybrid `0.0328` · `0.0320` · `0.0315`, Cohere `0.687` · `0.517` · `0.451`.

# Perché il cross-encoder non si fa su tutto il corpus

`82 chunk → 82 coppie per domanda`

Si fa. Il nostro corpus.

`1.000.000 chunk → 1.000.000 coppie per domanda`

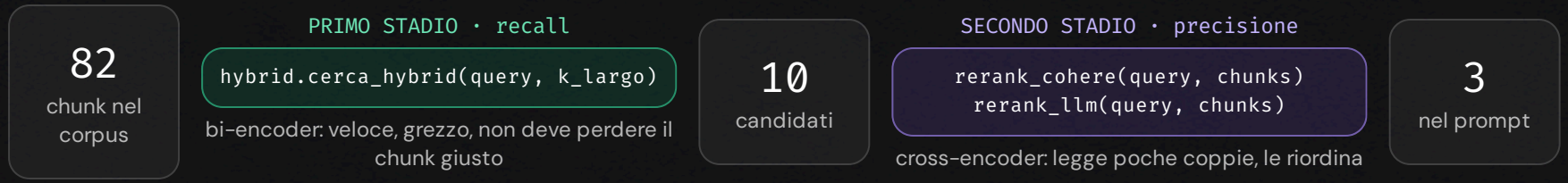
No. E niente da pre-calcolare: il punteggio dipende dalla domanda, che non c'è finché non arriva.

| | pre-calcolo | costo per domanda | precisione sui quasi-duplicati |
|---|---|---|---|
| **bi-encoder** | sì: i vettori dei chunk | 1 codifica + una ricerca di vicini | bassa: vettori quasi uguali |
| **cross-encoder** | no | N letture, una per coppia | alta: legge la coppia |

> Il cross-encoder da solo non è un motore di ricerca: è un **giudice** che legge poco e bene.



# Two-stage retrieval: recall largo, poi ordine fine




Il primo stadio garantisce il **recall** (il chunk giusto è nella lista), il secondo la **precisione** (è in testa). Poi i primi tre al prompt, come prima. In una funzione: `cerca_due_stadi(query, k_largo, k_finale, backend)`.

# Quanto largo il primo stadio: la scrivania

Il secondo stadio riordina **solo ciò che riceve**, e riceve la **lista fusa** dell'hybrid. «La scrivania non sta ferma, si muove tutta quando scrivo»: il chunk del manuale è **17°** nel semantico e **28°** in BM25.

`k_largo = 10`

nella lista fusa: –
dopo Cohere: –

in nessuna delle due liste

`k_largo = 20`

nella lista fusa: 25
dopo Cohere: –

solo nella lista semantica: un contributo, e resta fuori dai 20 che passano

`k_largo = 30`

nella lista fusa: 17
dopo Cohere: 9 · dopo l'LLM: 19

in tutte e due le liste: due contributi, e passa

Allargare non sposta di un posto alla volta: a 30 candidati il chunk entra **anche in BM25**, e RRF somma i due contributi come sulle query con codice della lezione scorsa.

Due lezioni in un esempio: il primo stadio deve essere **largo abbastanza**, e il rerank **non cura la parafrasi** («non sta ferma» contro «oscilla alla massima altezza» → Modulo 3 · Lezione 13).



# Il costo di allargare il primo stadio

Allargare costa al secondo stadio, che legge più coppie. Ma i due backend non costano allo stesso modo.

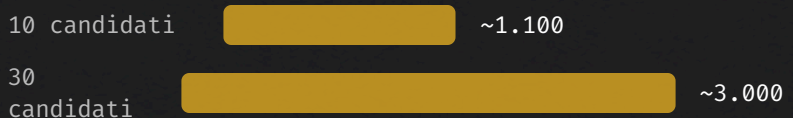
COHERE · search unit per chiamata

10 candidati — 1 unità

30 candidati — 1 unità

una unità copre fino a 100 documenti: 10 o 30 costano uguale

LLM · token in ingresso per chiamata




i token crescono con i candidati: il prompt li contiene tutti

> Stasera `k_largo = 10`: tiene dentro tutte le query con codice della lezione scorsa, e costa poco.

# Due backend per il secondo stadio




Stessa entrata, stessa uscita: cambia **chi legge le coppie**. Le due slide dopo li aprono.

# Cohere Rerank

UNA CHIAMATA

```
cohere_client.rerank(
  model="rerank-v3.5",
  query=query,
  documents=[c["testo"] for c in chunks],
  top_n=len(chunks),
)
```

Torna la lista in ordine di pertinenza, con un **relevance score** tra 0 e 1 per ogni chunk. Un cross-encoder addestrato apposta per questo compito.

- **≈ 1 s** a chiamata (0.3-3)
- **1 unità** fino a 100 documenti
- **stesso ordine** a ogni giro

I DATI ESCONO

I chunk viaggiano verso un servizio esterno, come già verso OpenAI per gli embedding. Se il corpus non può uscire, questa strada non c'è.



# L’LLM come reranker

IL PROMPT, E LA RISPOSTA STRUTTURATA

```
Domanda: Cosa copre SRV-02?

Passaggi numerati:
[0] Con il servizio SRV-03 estendi…
[1] SRV-01 — montaggio… SRV-02 — ritiro…
[2] TCK-1228 · SRV-01 · appuntamento…
…
Ordina TUTTI i passaggi dal più al meno utile.

→ Ordine(indici=[1, 0, 2, …])
```

`responses.parse(…, text_format=Ordine)`: lo structured output di Modulo 2 · Lezione 13. Il modello dà un **ordine**, non un punteggio.

≈ **2 s** a chiamata (0.9-3.6)

**~1.100 token** → $0.0002, crescono con i candidati

**ordine diverso** da un giro all'altro

NESSUN SERVIZIO NUOVO

La chiave OpenAI che avete già, il modello che avete già. Zero registrazioni.



# Cohere vs LLM: costo, latenza, privacy, stabilità

| criterio | Cohere Rerank | LLM come reranker |
|---|---|---|
| costo | 1 unità a chiamata, fino a 100 documenti · zero nel free tier | $0.0002 a 10 candidati · cresce con i candidati |
| latenza | ≈ 1 s (0.3–3) | ≈ 2 s (0.9–3.6) |
| privacy | i chunk escono verso Cohere | i chunk escono verso OpenAI (già succede per gli embedding) |
| stabilità | stesso ordine a ogni giro | l'ordine può cambiare da un giro all'altro |

I criteri sono quelli di Modulo 2 · Lezione 2 e 6, più uno: la **stabilità**.

> Terza strada: un cross-encoder open source in locale con `sentence-transformers`. I dati restano in casa, serve hardware. Si cita, non si usa.



# Quando il rerank non ripaga

IL PRIMO STADIO BASTA GIÀ

9 query su 13 della lezione scorsa erano già prime. Su quelle il secondo stadio costa e non cambia niente.

CORPUS PICCOLO

Se k può essere alto, il prompt regge tutti i candidati e il riordino è superfluo.

LATENZA STRETTA

Autocomplete, chat con budget di mezzo secondo: un secondo in più per domanda non ci sta.

Si accende **dopo aver misurato** che serve. È il lab della prossima lezione.



# Cosa costruiamo stasera: i pezzi e il prodotto

`cerca_due_stadi(query, k_largo, k_finale, backend)`

il prodotto finale: primo stadio → secondo stadio → i primi k_finale · **TODO 3**

`hybrid.cerca_hybrid()`

il primo stadio · già in `aikit/`, non si tocca

`rerank_cohere(query, chunks)`

il secondo stadio con Cohere · **già nel file**, codice spiegato

`rerank_llm(query, chunks)`

lo stesso lavoro con l'LLM · **TODO 2**, scrivete voi

stessa firma, stesso ritorno: il backend si sceglie con un `if`, il resto non cambia

Il codice segue quest'ordine: funzione 1, TODO 2, TODO 3.



# Codice spiegato: rerank_cohere()

**CODICE SPIEGATO — il docente spiega, voi seguite sul vostro file**

`working/rerank.py` · funzione 1, già scritta · 12 minuti

si rientra dopo il lancio, con la demo a schermo

COSA C'È NEL FILE

`CONFIG` (k_largo 10, k_finale 3, i due modelli, la pausa di 6 s) · `DOMANDA_DEMO` «Cosa copre SRV-02?» · `QUERY_SET` uguale alla lezione scorsa · i due client · il main

`1 · rerank_cohere(query, chunks)`

`rerank(model, query, documents=[i testi], top_n=len(chunks))` — solo i testi: il modello vede stringhe, non chunk; li rivogliamo tutti

→ `risposta.results`, già in ordine di pertinenza

→ ogni result ha `index` (la posizione nella lista mandata) e `relevance_score`

→ il ciclo rimette insieme il chunk originale e lo score nuovo, nella forma di `rag.recupera()`

La funzione è già nel vostro file: si segue, non si ricopia.



# Cosa si è visto: la demo su SRV-02

```
$ python working/rerank.py

D: Cosa copre SRV-02?
  [hybrid, primi 3 di 10]
    1. policy-resi-garanzia-009 score=   0.033  Con il servizio SRV-03 estend
    2. listino-catalogo-006     score=   0.032  SRV-01 — montaggio… SRV-02 —
    3. ticket-supporto-012      score=   0.032  TCK-1228 · SRV-01 · appuntamer
  [cohere, primi 3 di 10]  (0.54 s)
    1. listino-catalogo-006     score=   0.687  SRV-01 — montaggio… SRV-02 —
    2. policy-resi-garanzia-008 score=   0.517  Ogni prodotto LumaDesk è coper
    3. policy-resi-garanzia-009 score=   0.451  Con il servizio SRV-03 estend
```

DA SECONDO A PRIMO

Tre score appiccicati a sinistra, tre staccati a destra. Il chunk giusto era lì: il modello che legge la coppia lo ha riconosciuto, in mezzo secondo.

ORA IL SECONDO BACKEND

E i due stadi in una funzione: TODO 2-3.



# Esercizio: rerank_llm() e cerca_due_stadi()

**SCRIVETE VOI — da soli, con affiancamento**

`working/rerank.py` · `TODO 2-3` · `45 minuti` · `consegna in CONSEGNA.md`

si rientra per il reveal

`TODO 2 · rerank_llm(query, chunks)`

prompt con la domanda e i chunk numerati → `responses.parse(…, text_format=Ordine)` → i chunk nell'ordine degli indici, score = quanti ne restano dietro → stampa dei token. Ogni chunk **una volta sola**, e **tutti**.

`TODO 3 · cerca_due_stadi(query, k_largo, k_finale, backend)`

`hybrid.cerca_hybrid(query, k_largo)` → il backend scelto (`"cohere"` / `"llm"`) → i primi k_finale

`FATTO QUANDO`

1 · SRV-02, codice sconto e metà corsa (le tre che l'hybrid teneva seconde) sono **prime** col backend che hai scritto tu, e con Cohere se l'hai provato.

2 · Le query che l'hybrid aveva già prime **non escono dalla top-3**: se una cambia posto con l'LLM, non è un tuo errore.

3 · **Secondi e token** annotati nella tabella in fondo al file (la colonna del chunk giusto è già compilata).

Errori tipici e «se finisci prima»: in `CONSEGNA.md`.

# Reveal: la tabella

| # | query | chunk giusto | hybrid | cohere | llm |
|---|---|---|---|---|---|
| 1 | …ticket TCK-1152? | ticket-supporto-008 | 1 | 1 | 1 |
| 2 | Cosa copre SRV-02? | listino-catalogo-006 | 2 | 1 | 1 |
| 3 | …risolto il TCK-1201? | ticket-supporto-004 | 1 | 1 | 1 |
| 4 | Con l'errore E07… | manuale-lumadesk-009 | 1 | 1 | 1 |
| 5 | Quale RMA… danno da trasporto? | listino-catalogo-008 | 1 | 1 | 1 o 4-7 |
| 6 | …firmware 2.3.0? | changelog-rilasci-002 | 1 | 1 | 1 |
| 7 | La scrivania non sta ferma… | manuale-lumadesk-013 | — | — | — |
| 8 | Quanto tempo ho… contributo… | guida-smart-working-004 | 3 | 2 | 1 |
| 9 | Posso lavorare dall'estero? | guida-smart-working-009 | 1 | 1 | 1 o 2 |
| 10 | Il monitor scende piano piano… | ticket-supporto-007 | 1 | 1 | 1 |
| 11 | …codice sconto: posso restituire? | policy-resi-garanzia-007 | 2 | 1 | 1 |
| 12 | …si ferma sempre a metà corsa? | ticket-supporto-003 | 2 | 1 | 1 |
| 13 | …in contrassegno? (fuori corpus) | — | — | — | — |



# Cosa dice la tabella

**CURATE**

SRV-02 2 → 1, codice sconto 2 → 1, metà corsa 2 → 1, con entrambi i backend. Contributo 3 → 2 con Cohere, 3 → 1 con l'LLM. Il chunk giusto era in top-3: il secondo stadio lo ha letto e riconosciuto.

**NON CAMBIA**

Le otto che l'hybrid aveva prime restano prime con Cohere, a ogni giro. Il rerank non ha rotto niente.

**OSCILLA**

Con l'LLM la riga RMA e, di rado, l'estero cambiano posto da un giro all'altro: la slide dopo.

**NON SI CURA STASERA**

La scrivania: 17ª nel semantico, fuori dal primo stadio a 10 (→ Modulo 3 · Lezione 13). Il contrassegno: fuori corpus, il rifiuto lo dà `rispondi()`.



# Il reranker LLM oscilla

«Quale RMA devo aprire per un danno da trasporto?» — la stessa top-10, più giri per backend. Posizione del chunk giusto:




Quando scende, preferisce i chunk della policy resi al listino delle procedure RMA. «Posso lavorare dall'estero?»: 1 giro su 9 al secondo posto.

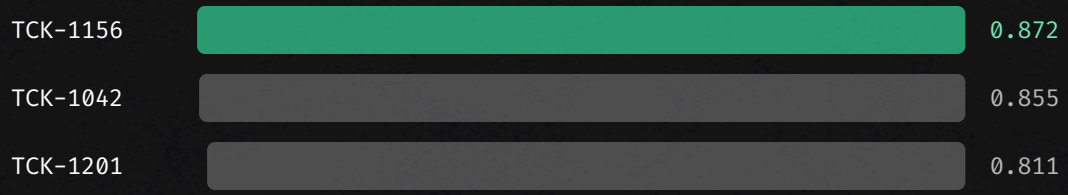
STABILITÀ: IL QUARTO CRITERIO

Sulla stessa lista, giri diversi danno ordini diversi. Il cross-encoder dedicato dà lo stesso ordine ogni volta. Accanto a costo, latenza e privacy, si sceglie anche per questo.

Chi ha in tabella un numero diverso dal vicino **non ha sbagliato**.

# Il cluster dei quasi-duplicati: 0.872 contro 0.855

«Perché il piano si ferma sempre a metà corsa?» — cinque ticket che iniziano tutti allo stesso modo. L'hybrid metteva primo TCK-1042, secondo il risolutivo TCK-1156.




relevance score di Cohere sulla top-10 dell'hybrid

```
TCK-1156 · il piano si ferma a metà corsa — … memoria di calibrazione corrotta … ciclo di reset completo

TCK-1042 · il piano si ferma a metà corsa — … la cassettiera sporgeva sotto il piano

TCK-1201 · il piano si ferma a metà corsa — … alimentatore non originale
```

> Ha vinto per **0.017**: anche il cross-encoder li vede vicini, perché sono vicini davvero. Sui testi quasi uguali il secondo stadio distingue quel poco che basta, non di più.

# Tempi e costi dei due backend

Dalla demo e dalle vostre tabelle, a 10 candidati.

COHERE RERANK

| ≈ 1 s | 1 | = |
|---|---|---|
| a chiamata (0.3-3) | search unit | a 30 candidati |

14 chiamate per giro (demo compresa): 14 unità su 1.000 al mese

LLM COME RERANKER

| ≈ 2 s | $0.0002 | ×3 |
|---|---|---|
| a chiamata (0.9-3.6) | ~1.100 token in, 11-25 out | a 30 candidati: ~3.000 token |

13 chiamate per giro: meno di mezzo centesimo in tutto

Un giro del query set: circa due minuti, quasi tutti di pausa per il free tier.

> **9 query su 13** erano già prime: su quelle il secondo stadio ha solo aggiunto un secondo.



# La prossima lezione

Le leve del retrieval ora ci sono tutte:

k · soglia · chunk size · prompt · **hybrid** · Modulo 3 · Lezione 4 · **rerank** · stasera

→ **Modulo 3 · Lezione 6 — Lab RAG: debugging del retrieval**
si va a caccia dei failure e si impara a capire **quale leva serve**, a partire dallo stadio rotto

Più avanti, a Modulo 3 · Lezione 10, la meccanica di stasera — un modello che legge domanda e testo e giudica — torna con un altro bersaglio: valutare le risposte.



# Grazie

prossima lezione: Modulo 3 · Lezione 6 — Lab RAG: debugging del retrieval

